

🔗 Generating a BIP39 Seed Phrase with a 20-Sided Die — Extended Edition: 12 & 24 Words

A BIP39 mnemonic seed phrase provides the master entropy for your Bitcoin wallet. Software can generate that randomness for you, but physical dice offer a transparent, verifiable source of entropy that does not depend on any electronic random-number generator. This extended edition covers the full 12-word procedure, the 24-word procedure, and the last-word-completion shortcut offered by several seed tools. It uses the same lookup table (on the back).

📖 Why verifiable entropy? — the July 2026 incident

In July 2026, more than a thousand BTC were swept within minutes from wallets whose seeds had been created by Coldcard devices carrying a firmware flaw dating back to 2021: seed generation silently used a predictable software PRNG instead of the hardware RNG, delivering far less entropy than promised. The vendor patched quickly — and its own advisory notes that seeds generated from 50 or more honest dice rolls were never at risk. The lesson is not about one brand: **all electronic entropy is invisible to its user**. You cannot look at a chip and see whether its randomness is healthy — but you can look at a die. With this tutorial, every bit of your seed comes from rolls you verify with your own eyes.

📊 Understanding the Table Structure

The reference table is organized as a three-dimensional lookup system. Each of the 2048 words of the BIP39 standard is uniquely identified by three coordinates, which we will call D1, D2 and D3. The first coordinate (**D1**) selects one of the **8 page sections**, the second (**D2**) selects one of the **16 rows** within that section, and the third (**D3**) selects one of the **16 columns**. Together, these three values identify exactly one word in the table. Seed words are always those of the English BIP39 standard, accepted by virtually every wallet.

🎲 The Rolling Procedure (12 Words)

For each of the first 11 words of your seed phrase, you must obtain three valid die results. Roll the die and record the result only if it shows a value between 1 and 16 (inclusive). If the die shows 17, 18, 19 or 20, **discard that roll** and roll again until you obtain a valid result. Repeat this process until you have three valid numbers (D1, D2, D3) for the current word.

💡 Word Generation Example

Suppose you are generating the fifth word of your seed. You roll the die and obtain: 19 (invalid, roll again), 3 (valid, D1 = 3,4), 1 (valid, D2 = 1), 20 (invalid, roll again), 11 (valid, D3 = 11). Your coordinates are ((3,4), 1, 11). Navigate to section 3,4 of the table, find row 1 and then column 11 to obtain your word: **candy**.

🔑 The 12th Word

To generate the last word of your BIP39 seed, repeat the procedure for the first 2 coordinates exactly as done for the first 11 words, thereby selecting the section and the row of the table. At this point, there is exactly one word in the selected row (out of 16 alternatives) that will form a valid BIP39 seed. The column of the last word is **not** determined by rolling dice, but by trial and error in your wallet's seed-entry wizard — preferably on an *air-gapped* device: it will reject the 15 invalid candidates and accept the single valid one.

💡 Last Word Generation Example

Suppose your first 11 words are, in order: *“never, use, this, example, because, private, key, secret, need, phrase, and true”*. Now you will generate your 12th seed word. You roll the die and obtain: 18 (invalid, roll again), 12 (valid, D1 = 11,12), 9 (valid, D2 = 9). With that, your 12th word is already determined: test the words of row 9 of section 11,12 one by one and you will see that the word of column 14, **random**, (and only it), together with the first 11, in this order, forms a valid BIP39 seed:

1.never 2.use 3.this 4.example 5.because 6.private 7.key 8.secret 9.need 10.phrase 11.true 12.random. 🟢

🔗 Shortcut: Wizards with Last-Word Completion

Several seed tools can complete the last word for you: given all previous words, they list exactly the candidates that make the checksum valid — for example SeedSigner's *final word* flow, the SeedPicker last-word calculator, Coinplate's offline BIP39 Recoverer, or Ian Coleman's BIP39 page (used offline). For a 12-word seed there are always exactly **128 valid last words: precisely one in each of the $8 \times 16 = 128$ rows** of the table. So instead of column-by-column trial and error: roll D1 and D2 as usual, then take, among the wizard's candidates, **the one lying in your rolled section and row**. Same word, same entropy, fewer keystrokes. This is no accident of our example: *whatever* the first 11 words, once D1 and D2 are fixed, one and only one word of that row satisfies the checksum — the repository's sanity tests verify it mechanically over random prefixes.

24 Words

For a 24-word seed (256 bits of entropy), generate words 1–23 exactly like the first 11 words above: three valid rolls each ($23 \times 11 = 253$ bits). For the **24th word**, roll only **D1**, selecting the section (3 more bits — 256 in total). The row and the column are then fixed by the 8-bit checksum: **exactly one word in your entire 256-word section** completes a valid seed. Find it in one of two ways:

- **Trial and error** in the wallet wizard, across the rolled section (up to 256 attempts — tedious, but fully offline); or
- **Last-word completion**: a wizard as above will offer exactly **8 valid candidates** — **one per section**. Your D1 roll picks the section; take the candidate lying in it.

Likewise guaranteed, not coincidental: *whatever* the first 23 words, once D1 is fixed, one and only one word of that section satisfies the checksum — also machine-verified by the repository’s sanity tests.

24-Word Example: the Example Phrase, Duplicated

Take the 12-word example above and duplicate it to obtain words 1–23; for the 24th, suppose you roll: 17 (invalid, roll again), 11 (valid, D1 = 11,12). Among the wizard’s 8 candidates (or by trial and error across section 11,12), exactly one word of that section — **polar**, row 4, column 12 — completes a valid 24-word seed:

never use this example because private key secret need phrase true random
never use this example because private key secret need phrase true polar ✓

Note the sacrifice: the duplication cannot keep **random** as the 24th word, because the 8-bit checksum of the full 256 bits differs from the 4-bit checksum that made **random** valid at 12 words. The checksum claims the last word — and lands on **polar**, the one word in the *same section* (D1 = 11,12) as **random** whose combination with the previous 23 satisfies it.

Summary

- Roll a 20-sided die. **Discard** results **17, 18, 19** or **20**
- Each valid roll (1 through 16 inclusive) specifies one coordinate of one word
- Each of the **8 sections (D1)** corresponds to **2** possible valid results; each of the **16 rows (D2)** and **16 columns (D3)** to **1**
- **12 words**: 11 words fully rolled, then D1 + D2 only; **35 valid rolls** give the full **128 bits**. The last word: trial and error in the row, or the wizard candidate in your rolled row (**128** valid candidates exist, one per row)
- **24 words**: 23 words fully rolled, then D1 only; **70 valid rolls** give the full **256 bits**. The last word: trial and error in the section, or the wizard candidate in your rolled section (**8** valid candidates exist, one per section)
- The checksum bits (4 or 8) are always the wallet’s job — never rolled

Security Warnings

- Never use the seed phrase from this example above: because, in order for your private keys to be secret, you need your seed phrase to be true random
- Perform this procedure in a private place. Do not photograph or digitally record your rolls or intermediate results. Write your final seed phrase on paper and store it in a safe place
- Only run last-word-completion tools on trusted *air-gapped* devices — they see your whole seed. For maximum security, only ever type your seed phrase into air-gapped devices that you own
- If you suspect one of your seeds was generated by flawed software or firmware (as in the July 2026 incident), generate a fresh seed with this method and move your funds to it: a firmware update does **not** repair an existing weak seed

Learn More

This tutorial is by **Yuri da Silva Villas Boas** — author of BIP-450 (Formosa) and creator of the Great Wall protocol. Official repository (sources, PDFs in 4 languages, automatic verification): github.com/Yuri-SVB/BTC-D20 — or scan the QR code on the side.

Short paper: why a quiet fix was impossible — Kerckhoffs and the July 2026 incident: [posts/kerckhoffs-lemma-coldcard.md](#) in the repository

More tutorials, courses and community: www.loudproudandfree.com — and follow us on Twitter/X, Nostr and Telegram

Great Wall: Generating a strong seed is only step one; protecting it against theft and coercion is the rest. Great Wall is our free-software protocol for coercion-resistant self-custody: github.com/Yuri-SVB

